Intelligence Artificielle 1

Cours 2 et 3 - Introduction Python

Steve Lévesque, Tous droits reservés © où applicables

Table des matières

1. Aspects importants de Python
   - Syntax (indentation)
   - Typage et casting
2. Variables et structures de données (tableaux, dictionnaires)
3. Fonctions
   - Appels de fonction
   - Les paramètres et arguments
4. Documentation du code (commentaires)
5. Importation de fonction dans des fichiers
6. Exercice formatif 2
7. Erreurs (compilation et logique)
   - Exceptions (compilation)
   - Tests unitaires (logique)
8. Manipulation de structure de données de base
9. Utilisation de modules externes populaires "open-source"
   - Exemple 1 - Module Numpy
   - Exemple 2 - Module Sklearn
10. Exercice formatif 3

Syntax (indentation)

Python fonctionne sur la base de l'indentation pour la structuration ainsi que la validation du code.

Contrairement aux autres langages populaires, une mauvaise indentation/structure du code va causer une erreur de compilation ou des mauvaises exécutions.

*S'applique aussi aux fonctions.

Syntax (indentation)

Listing: https://www.w3schools.com/python/python_syntax.asp

```python
# Steve Levesque, All rights reserved

# Incorrect
if 2 > 1:
print("2 is larger than 1 : True")

# Correct
if 2 > 1:
    print("2 is larger than 1 : True")
```

Typage et casting

Le type est enforcé en Python. Alors il faut bien les gérer lorsque l'on programme.

Généralement, les langages sont typés lorsque leur but demande beaucoup de précision.

Par exemple, dans un langage typé, il n'est pas possible de concaténer un chiffre (int, float, etc.) avant de l'avoir converti en chaîne de caractères (string).

Typage et casting

Listing: https://www.w3schools.com/python/python_datatypes.asp

```
# Steve Levesque, All rights reserved

"""
    print("La note de passage est : " + 60)
          ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
TypeError: can only concatenate str (not "int") to str
"""
print("La note de passage est : " + 60)
```

Typage et casting

Listing: https://www.w3schools.com/python/python_datatypes.asp

```
1  # Steve Levesque, All rights reserved
2
3  """
4  La note de passage est : 60
5  """
6  print("La note de passage est : " + str(60))
```

Variables et structures de données - Tableaux

Les listes sont utilisées pour stocker plusieurs éléments dans une seule variable.

Les listes sont l'un des 4 types de données intégrées en Python utilisées pour stocker des collections de données, les 3 autres sont **Tuple**, **Set** et **Dictionary**, tous avec des qualités et des utilisations différentes.

Les listes sont créées à l'aide de crochets carrés : "[]"

Variables et structures de données - Tableaux

Listing: https://www.w3schools.com/python/python_lists.asp

```python
# Steve Levesque, All rights reserved

# Base declaration
grades = [60, 80, 90, 80]

# Print directly
print(grades)

# Convert to string and print with concat to another string sentence
print("The array itself as a string : " + str(grades))

# Iterative print for each element in array (in order)
print()
print("Print each element of the array with a special loop : ")
for grade in grades:
    print(grade)
```

Variables et structures de données - Dictionnaires

Les dictionnaires sont utilisés pour stocker les valeurs de données dans des paires clé/valeur.

Un dictionnaire est une collection ordonnée*, modifiable et qui n'autorise pas les doublons.

Les dictionnaires sont écrits avec des accolades "{}", et ont des clés et des valeurs.

*Depuis la version 3.7 de Python, les dictionnaires sont ordonnés. Dans Python 3.6 et les versions antérieures, les dictionnaires ne sont pas ordonnés.

Variables et structures de données - Dictionnaires

Listing: https://www.w3schools.com/python/python_dictionaries.asp

```python
# Steve Levesque, All rights reserved

# "key": "value"
example =     {
  "brand": "Ford",
  "electric": False,
  "year": 1964,
  "year": 2020, # Will replace 1964 with 2020 since duplicate
  "colors": ["red", "white", "blue"]
}

# {'brand': 'Ford', 'electric': False, 'year': 2020,
# 'colors': ['red', 'white', 'blue']}
print(example)
```

Fonctions

Une fonction est un bloc de code qui ne s'exécute que lorsqu'elle est appelée.

Vous pouvez transmettre des données, appelées paramètres, à une fonction.

Une fonction peut renvoyer des données en conséquence.

Composantes d'une fonction :

- Appels de fonction
- Les paramètres (variables listées) et les arguments (les valeurs réelles passées à la fonction) :
  - Le nombre de paramètres/arguments
  - Les valeurs par défaut
  - Le type du paramètre

Variables et structures de données - Dictionnaires

Listing: https://www.w3schools.com/python/python_functions.asp

```python
# Steve Levesque, All rights reserved

# Declare the function
def addition():
    print("Simple function call without values...")


# Call the function to execute the code inside it.
addition()
```

Les paramètres et arguments

Listing: https://www.w3schools.com/python/python_functions.asp

```
# Steve Levesque, All rights reserved

# Declare the function
# The default values of "first" and "second" are equal 1 when no arguments
def addition(first=1, second=1):
    # Return gives the result back to the function call.
    return first + second


# Call the function to execute the code inside it.
# When no values (arguments) are passed, the parameters
# are by default from the specified values in the function declaration.
print(addition())
```

Les paramètres et arguments

Listing: https://www.w3schools.com/python/python_functions.asp

```python
# Steve Levesque, All rights reserved

# Declare the function
# The default values of "first" and "second" are equal 1 when no arguments
def addition(first=1, second=1):
    # Return gives the result back to the function call.
    return first + second


# Call the function to execute the code inside it.
# With values (args) specified, the function will yield the result.
# "5"
print(addition(2, 3))
```

Les paramètres et arguments

```python
# Steve Levesque, All rights reserved

"""
addition
Return the added total of an array of numbers.

numbers [List]: A list of numbers
"""
def addition(numbers):
    res = 0

    for num in numbers:
        res += num

    return res

# Call the function to execute the code inside it.
nums = [1, 3, 5]
print(addition(nums))
```

Documentation du code (commentaires)

En Python la documentation est simple et rapide.

Il y a deux types de commentaires :

- Par ligne (unitaire) : avec le “hashtag” (#)
- Par bloc (multiple : avec les guillemets simples ou doubles en suite de 3 (”’ ou ”””)

Documentation du code (commentaires)

Listing: https://www.w3schools.com/python/python_comments.asp

```
# Steve Levesque, All rights reserved

"""
A comment in block
1
2
"""

# Comment for singular lines
# 1

"""
    This is a wrong comment evaluated as code...
              ^^^^^
SyntaxError: invalid syntax
"""
This is a wrong comment evaluated as code...
```

Importation de fonction dans des fichiers

Pour organiser le code dans un gros projet et/ou faire des tests unitaires, il est très pertinent de savoir comment importer des fonctions d'un fichier externe dans un nouveau fichier.

Importation de fonction dans des fichiers

```
1  # Steve Levesque, All rights reserved
2
3  def addition(numbers):
4      res = 0
5
6      for num in numbers:
7          res += num
8
9      return res
```

Importation de fonction dans des fichiers

```
1  # Steve Levesque, All rights reserved
2
3  #    filename          function name
4  from calculator import addition
5
6
7  # Import the function from the file and use it as it is
8  # "11"
9  print(addition([3, 6, 1, 1]))
```

Exercice formatif 2

Voir l'énoncé dans la remise sur Léa.

Erreurs (compilation et logique)

Puisque le code est écrit par des humains (ou maintenant par l'intelligence artificielle), celui-ci est sujet aux erreurs.

Il est possible de s'équiper avec des méthodes pour empêcher deux types d'erreurs courantes :

- Compilation : le code (une partie) n'exécute pas du tout et on voudrait quand même continuer l'exécution du programme
  - Solution : Exceptions
- Logique : le code exécute, mais le retour voulu est erroné
  - Solution : Tests unitaires

Exceptions (compilation)

En Python, il est possible de gérer les exceptions avec “try” et “expect”.

Les erreurs de compilation peuvent survenir de manière planifiée dans certains cas et seulement une gestion d’exception est la solution pour résoudre le problème.
Par contre, il est généralement non recommandé d’utiliser trop souvent un gestionnaire d’exception et de trouver une solution à la source lorsque possible.

Exceptions (compilation)

Listing: https://www.w3schools.com/python/python_comments.asp

```
# Steve Levesque, All rights reserved

# The variable "x" is not defined.
# In a normal execution, there would be a compilation error
# and the program would stop executing.
try:
  print(x)
except:
  print("An exception occurred")

# Result:
# An exception occurred
```

Tests unitaires (logique)

Si un programme retourne une valeur erronée, celui-ci compile tout de même. L'ordinateur (le compilateur) n'a pas de compréhension vis-à-vis la logique du code à son domaine d'application.

Tests unitaires (logique)

```python
# Steve Levesque, All rights reserved
import unittest
from calculator import addition

class TestCalculatorAddition(unittest.TestCase):

    def test_add_from_array(self):
        nums = [1, 3, 4]
        res = 8
        self.assertEqual(addition(nums), res)

    def test_add_from_array_large(self):
        nums = [1, 3, 4, 1, 1, 2, 3, 4, 5, 6, 7, 1, 3]
        res = 41
        self.assertEqual(addition(nums), res)

if __name__ == '__main__':
    unittest.main()
```

Tests unitaires (logique)

Si la fonction est mauvaise, voici un log avec les erreurs et de l'information additionnelle pertinente.

```
# Steve Levesque, All rights reserved
"""
FAIL: test_add_from_array (__main__.TestCalculatorAddition.test_add_from_array)
----------------------------------------------------------------------
Traceback (most recent call last):
    self.assertEqual(addition(nums), res)
AssertionError: -8 != 8

======================================================================
FAIL: test_add_from_array_large (__main__.TestCalculatorAddition.test_add_from_
----------------------------------------------------------------------
Traceback (most recent call last):
    self.assertEqual(addition(nums), res)
AssertionError: -41 != 41

----------------------------------------------------------------------
Ran 2 tests in 0.001s

FAILED (failures=2)
"""
```

Manipulation de structure de données de base (tableaux, dictionnaires)

Dans tous les langages, il est normalement possible de faire des opérations CRUD sur les structures de données.

Python supporte de manière très intuitive les opérations CRUD :

- (C)reate
- (R)ead
- (U)pdate
- (D)elete

Les tableaux et dictionnaires seront abordés.

Manipulation de structure de données de base - Tableaux

```python
# Steve Levesque, All rights reserved

crud_list = []

# [C]reate
crud_list.append(1)
print(crud_list)

# [R]ead
print(crud_list[0])

# [U]pdate
crud_list[0] = 42
print(crud_list)

# [D]elete
del crud_list[0]
print(crud_list)

# Results: [1] 1 [42] []
```

Manipulation de structure de données de base - Dictionnaires

```python
# Steve Levesque, All rights reserved

crud_dict = {}

# [C]reate
crud_dict["banana"] = "red"
print(crud_dict)

# [R]ead
print(crud_dict["banana"])

# [U]pdate
crud_dict["banana"] = "yellow"
print(crud_dict)

# [D]elete
del crud_dict["banana"]
print(crud_dict)

# results: {'banana': 'red'} red {'banana': 'yellow'} {}
```

Utilisation de modules externes populaires "open-source" pour aider à la programmation en AI

Il suffit d'installer le module en suivant les étapes suivantes :

- Rechercher "npm (nom du module, i.e. numpy)" sur votre navigateur favori.
- Cliquer sur le premier résultat qui apparait en lien avec le module (en norme) : le site `www.npmjs.com/search?q=numpy`
- Copier la commande d'installation sur le site : i.e. "npm i numpy" pour cet exemple
- Aller dans la console de votre IDE avec le .venv d'activé
- Écrire la commande copiée précédemment, attendre, et le module s'installe tout seul

NB : Il est pertinent de faire comme ceci puisque les noms des modules ne sont pas toujours simples et intuitifs dans le registre npm.

Exemple 1 - Module Numpy

```
# Steve Levesque, All rights reserved

"""
(.venv) PS D:Steve> pip install numpy
Collecting numpy
  Obtaining dependency information for numpy from https://files.pythonhosted.org
  Downloading numpy-1.26.4-cp312-cp312-win_amd64.whl.metadata (61 kB)
     ---------------------------------------- 61.0/61.0 kB 819.4 kB/s eta 0:00:0
Downloading numpy-1.26.4-cp312-cp312-win_amd64.whl (15.5 MB)
   ---------------------------------------- 15.5/15.5 MB 3.6 MB/s eta 0:00:00
Installing collected packages: numpy
Successfully installed numpy-1.26.4

[notice] A new release of pip is available: 23.2.1 -> 24.0
[notice] To update, run: python.exe -m pip install --upgrade pip
"""
```

Module - Numpy

```
# Steve Levesque, All rights reserved

# The module installed earlier, made by experts and open-source
import numpy as np

arr = np.array([8, 7, 6, 1, 2, 3, 4, 5])
arr = np.sort(arr)

# [1 2 3 4 5 6 7 8]
print(arr)
```

Exemple 2 - Module Sklearn

Listing:
https://scikit-learn.org/stable/auto_examples/classification/plot_digits_classification.html

```
# Author: Gael Varoquaux <gael dot varoquaux at normalesup dot org>
# License: BSD 3 clause
from sklearn import datasets, metrics, svm
from sklearn.model_selection import train_test_split

# MNIST Project : load database and flatten images
digits = datasets.load_digits()
data = digits.images.reshape((len(digits.images), -1))

# Create a classifier: a support vector classifier
clf = svm.SVC(gamma=0.001)
# Split data into 50% train and 50% test subsets
X_train, X_test, y_train, y_test = train_test_split(
    data, digits.target, test_size=0.5, shuffle=False
)

# Learn the digits on the train subset
clf.fit(X_train, y_train)
# Predict the value of the digit on the test subset
predicted = clf.predict(X_test)
```

Exercice formatif 3

Voir l'énoncé dans la remise sur Léa.

Bibliographie

- https://www.w3schools.com/python/
- https://www.npmjs.com
- https://scikit-learn.org/stable/auto_examples/classification/plot_digits_classification.html